ECEN 749 Lab 2 - Report

Yashas Bharadwaj
UIN: 635009332; Section: 601

Introduction

This lab is to create a MicroBlaze processor system using Vivado block designer. After creating the processor with GPIO capabilities by using IP block, we were given a code that when a push button was pressed the LEDs displayed up-counter and the same counter value was displayed on the TCL console as well. The exercise given to us was to use the functionality of the push buttons and the switches to display either counter decrement or increment or to display the position of the Switch on the console. This lab is purely a software lab and we had to analyse whether using HDL was easier as with the previous lab or with a software code with a hardware wrapper.

Procedure

1. We had to create a MicroBlaze processor system block design. This was done by adding IP blocks. The first two blocks that were added were Debug and UART, with 64KB and AXI ports enabled. The clocking option needed to be set as a Single ended clock capable pin from the differential clock option. This option was selected since we want just a single clock source from the FPGA and the return path is usually grounded. A GPIO block was added to interact with the LED, the width of this block was set to 4 since we have 4 LEDs. Reset_rtl_o had to be replaced by Constant IP renamed as VDD. After this, the design needed to be validated and then to be created as a HDL wrapper after which Bitstream was generated and then the hardware was exported as a .xsa file to Vitis IDE. Building C code will generate a binary which needs to be flashed onto Zybo board to see the output. Along with this the TCL console also needs to be launched using the run as a configuration button.
2. For the exercise portion of the lab, an additional IP block needs to be added configured as a GPIO for switches and push buttons needed for the experiment. This block has 8 inputs with a width of 8 bits. The .xdc file needs to be modified to include push buttons and switches. The rest of the procedure is the same as before.

Results

1. For the first lab experiment i.e. the example code, there are 4 LEDs that were glowing from 0 to 15 while incrementing, the value of the counter was also printed on the console. THe C code introduces the Vitis Driver APIs, this includes XGpio_Initialize (this is used to initialize the LEDs and the GPIOs), XGpio_SetDataDirection (this is to set the LED as output), XGpio_SetDiscreteWrite (this is to display the values on the LEDs from the switch state). An important thing to note in this code is that the int count variable is of the type u32 which means that count_masked is used which ensures only the last 4 bits are used.

2. For the exercise, the 8 bit input GPIO had to be initialized too using the same XGpio_Initialize command. Along with the previously mentioned functions we also have to use the XGpio_DiscreteRead command for reading the state of the switches and push buttons. In the code, there is an infinite while loop running, inside which there are “if else” statements for various cases for different requirements. The push buttons were selected by masking the 8 bits with 0x10, 0x20, 0x40, and 0x80, so that only one push button can be pressed at once for a particular operation. The first two push buttons were for counter increment and decrement respectively. And the push button 3 was for displaying the switch state using the LEDs and also printing the status on the console. Even here the masking was done to select a particular switch out of the upper four bits. The fourth push button when pressed should just display the count value. Both these codes use a custom delay function.

Conclusion

This lab taught us how to create a MicroBlaze processor system and interfacing it with the FPGA board using C language with the on board peripherals like push buttons, switches, and LEDs. We got to do this using custom IP blocks and Vitis Driver API.

Questions

a) In the previous lab we used a clock divider logic to basically divide the 125MHz to 1Hz frequency. This means that if a counter was used then it would could 125 Million times ! This basically means that our eyes would see it as just a continuously ON LED. So we had to keep the duty cycle to 50% and that was done by keeping it ON for 125/2 Million cycles and OFF for the other half. In this lab, the count is lower because each repetition requires a lot of cycles. The CPU operates at 100MHZ, and the delay is a C function, which uses around 10 cycles per instruction in the assembly code. With the count growing at 1Hz, which is equivalent to 100 million clock cycles at 100MHz, the divider is set to around 10 Million, which matches the amount used in the C code.

b) Volatile is a quantifier in C used to declare a variable that will tell the compiler that the value of this variable might change abruptly without any accompanying code. This stops the compiler from applying optimizations which could cause a problem with the while loop.

c) It means that the loop runs indefinitely. It executes all the commands included inside of it, whether reading input, performing some logical executions and displaying the output on the console.

d) The current lab was basically software based hardware implementation using C language and the last lab was purely hardware implementation using Verilog. I feel both labs were equally easy but if I had to choose, the second lab was much easier since the syntaxes of C were more readable and most of the libraries and header files were autogenerated for us. And displaying values on the console was also an advantage over the last lab. But the only disadvantage is that execution is slower.

Appendix

C code:

```
#include <xparameters.h>
#include <xgpio.h>
#include <xstatus.h>
#include <xil_printf.h>

/* Definitions */
#define GPIO_DEVICE_ID_LED XPAR_LED_DEVICE_ID     /* GPIO device that LEDs are connected to */
#define GPIO_DEVICE_ID_GPIO XPAR_GPIO_0_DEVICE_ID  //GPIO device ID for switches and push buttons
#define WAIT_VAL 10000000

int delay(void);

int main()
{
    int count;
    int count_masked;
    XGpio leds;
    int status_led, status_gpio;
    XGpio gpio;
    int state_gpio, state_led;

    status_led = XGpio_Initialize(&leds, GPIO_DEVICE_ID_LED);
    status_gpio = XGpio_Initialize(&gpio, GPIO_DEVICE_ID_GPIO);

    XGpio_SetDataDirection(&gpio, 1, 0xFF);

    XGpio_SetDataDirection(&leds, 1, 0x00);

    if (status_led != XST_SUCCESS) {
        xil_printf("Initialization failed\n");
        return XST_FAILURE;
    }

    if (status_gpio != XST_SUCCESS){
        xil_printf("SWITCHES and PUSH BUTTONS initialization failed\n");
        return XST_FAILURE;
    }
```

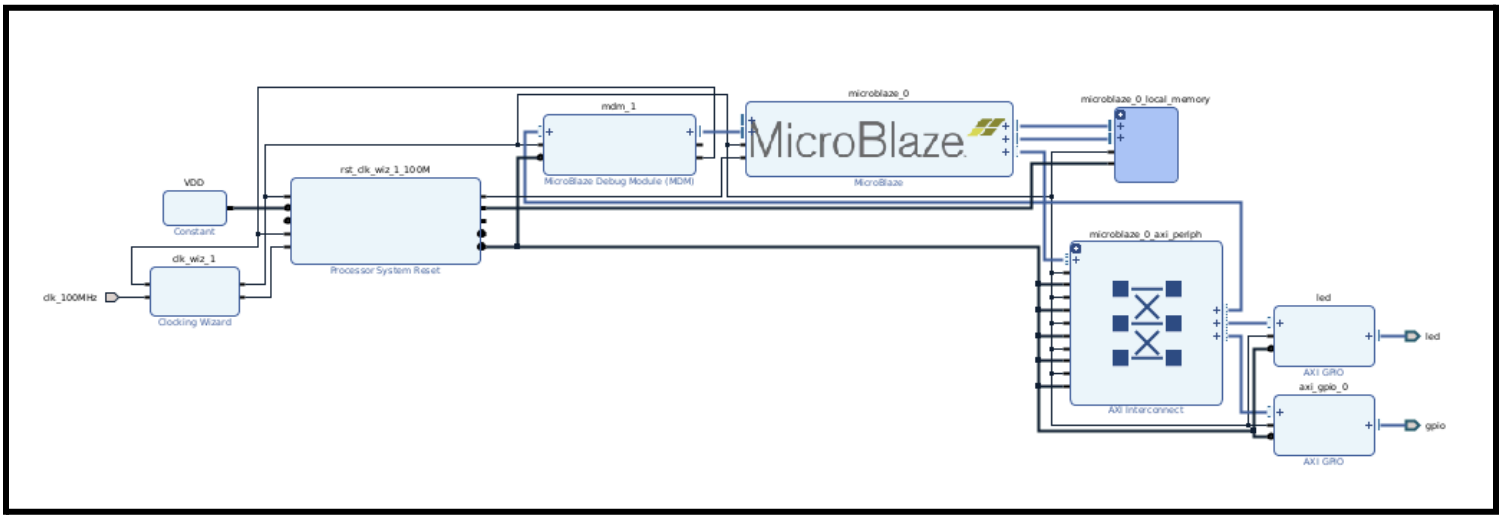
```
count = 0;

while (1)
{
    //read the state of the SWITCH and PUSH BUTTONS on the fpga

    state_gpio = XGpio_DiscreteRead(&gpio, 1);


    // check if the push button 0 is pressed or not
    if ((state_gpio & 0x10)){
        count++;            //increment the counter
        count_masked = count & 0xF;   //keep only lower four bits
        xil_printf("Push button 0 is pressed, value of the counter = 0x%x\n\r", count_masked);
    }


    //check if push button 1 is pressed or not
    else if ((state_gpio & 0x20)){
        count--;            //decrement the counter
        count_masked = count & 0xF;
        xil_printf("Push button 1 is pressed, value of the counter = 0x%x\n\r", count_masked);
    }

    //check if push button 2 is pressed or not
    else if ((state_gpio & 0x40)){
        int switch_state = XGpio_DiscreteRead(&gpio, 1);    //read the switch state
          state_led = switch_state & 0xF;              //since the switches are upper bits, mask the
lower bits

        XGpio_DiscreteWrite(&leds, 1, state_led);        //write the switch state to the leds
        xil_printf("When button 2 is pressed, value of the switch = 0x%x\n\r", state_led);

    }

    else if ((state_gpio & 0x80)){
        //count;        //display only count value
        count_masked = count & 0xF;        //lower 4 bit for the counter
          XGpio_DiscreteWrite(&leds, 1, count_masked);        //displays the counter value to the
leds
          xil_printf("when push button 3 is pressed, the value of count is displayed = 0x%x\n\r",
count_masked);
    }
```

```
        delay();

    }

    return 0;
}

int delay(void)
{
    volatile int delay_count = 0;
    while (delay_count < WAIT_VAL)
        delay_count++;
    return 0;
}
```

XDC file:

```
# CLOCK RTL
set_property PACKAGE_PIN K17 [get_ports clk_100MHz]
set_property IOSTANDARD LVCMOS33 [get_ports clk_100MHz]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk_100MHz]


# switches
set_property PACKAGE_PIN G15 [get_ports {gpio_tri_i[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_tri_i[0]}]

set_property PACKAGE_PIN P15 [get_ports {gpio_tri_i[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_tri_i[1]}]

set_property PACKAGE_PIN W13 [get_ports {gpio_tri_i[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_tri_i[2]}]

set_property PACKAGE_PIN T16 [get_ports {gpio_tri_i[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_tri_i[3]}]



#Push buttons
set_property PACKAGE_PIN K18 [get_ports {gpio_tri_i[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_tri_i[4]}]

set_property PACKAGE_PIN P16 [get_ports {gpio_tri_i[5]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_tri_i[5]}]

set_property PACKAGE_PIN K19 [get_ports {gpio_tri_i[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_tri_i[6]}]

set_property PACKAGE_PIN Y16 [get_ports {gpio_tri_i[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_tri_i[7]}]



# LED TRIO (Assuming LED0 to LED3 on Zybo Z7-10)
set_property PACKAGE_PIN M14 [get_ports {led_tri_o[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_tri_o[0]}]

set_property PACKAGE_PIN M15 [get_ports {led_tri_o[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_tri_o[1]}]

set_property PACKAGE_PIN G14 [get_ports {led_tri_o[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_tri_o[2]}]

set_property PACKAGE_PIN D18 [get_ports {led_tri_o[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led_tri_o[3]}]
```



lab2 - led_counter_sw/src/lab2_new.c - Vitis IDE

File Edit Search Vitis Project Window Help

Console | Problems | Vitis Log | Guidance

TCF Debug Virtual Terminal - MicroBlaze Debug Module at USER2

```
Push button 1 is pressed, value of the counter = 0xE
Push button 1 is pressed, value of the counter = 0xD
Push button 1 is pressed, value of the counter = 0xC
Push button 1 is pressed, value of the counter = 0xB
Push button 1 is pressed, value of the counter = 0xA
Push button 1 is pressed, value of the counter = 0x9
Push button 0 is pressed, value of the counter = 0xA
Push button 0 is pressed, value of the counter = 0xB
Push button 0 is pressed, value of the counter = 0xC
Push button 0 is pressed, value of the counter = 0xD
Push button 0 is pressed, value of the counter = 0xE
Push button 0 is pressed, value of the counter = 0xF
Push button 0 is pressed, value of the counter = 0x0
Push button 0 is pressed, value of the counter = 0x1
Push button 0 is pressed, value of the counter = 0x2
Push button 0 is pressed, value of the counter = 0x3
Push button 0 is pressed, value of the counter = 0x4
Push button 0 is pressed, value of the counter = 0x5
Push button 0 is pressed, value of the counter = 0x6
Push button 0 is pressed, value of the counter = 0x7
Push button 0 is pressed, value of the counter = 0x8
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x2
When button 2 is pressed, value of the switch = 0x2
When button 2 is pressed, value of the switch = 0x2
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x4
When button 2 is pressed, value of the switch = 0x4
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x1
When button 2 is pressed, value of the switch = 0x1
When button 2 is pressed, value of the switch = 0x1
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x8
When button 2 is pressed, value of the switch = 0x8
When button 2 is pressed, value of the switch = 0x8
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x0
when push button 3 is pressed, the value of count is displayed = 0x8
when push button 3 is pressed, the value of count is displayed = 0x8
when push button 3 is pressed, the value of count is displayed = 0x8
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x2
When button 2 is pressed, value of the switch = 0x0
When button 2 is pressed, value of the switch = 0x0
```